

USECASE 3 IHL

FINE TUNNING:

```
# =====
```

```
# SAMPLE QUANTUM CHATBOT USING FINE-TUNING
```

```
# Google Colab - Single Code Cell
```

```
# =====
```

```
# 1. Install required libraries
```

```
!pip install -q transformers datasets accelerate torch
```

```
# =====
```

```
# 2. Import libraries
```

```
# =====
```

```
from datasets import Dataset
```

```
from transformers import (
```

```
    AutoTokenizer,
```

```
    AutoModelForCausalLM,
```

```
    TrainingArguments,
```

```
    Trainer,
```

```
    DataCollatorForLanguageModeling
```

```
)
```

```
import torch
```

```
print("Libraries loaded successfully!")
```

```
# =====
```

```
# 3. Create a SMALL training dataset
```

```
# =====
```

```
training_data = [
```

```
    "Student: What is a qubit?\nTutor: A qubit is the basic unit of quantum information. It can  
represent quantum states using the principles of superposition.",
```

```
    "Student: What is superposition?\nTutor: Superposition means that a quantum system can  
exist in a combination of possible states until it is measured.",
```

```
    "Student: What is quantum entanglement?\nTutor: Quantum entanglement is a quantum  
phenomenon where two or more qubits have correlated quantum states.",
```

```
    "Student: What is a quantum gate?\nTutor: A quantum gate is an operation that changes the  
state of one or more qubits.",
```

```
    "Student: What is a Hadamard gate?\nTutor: The Hadamard gate creates a superposition of  
the computational basis states 0 and 1.",
```

```
    "Student: What is a CNOT gate?\nTutor: A CNOT gate is a two-qubit gate that flips the target  
qubit when the control qubit is in the state 1.",
```

```
    "Student: What is quantum measurement?\nTutor: Quantum measurement obtains  
information from a quantum state and produces a classical result.",
```

"Student: What is a quantum circuit?\nTutor: A quantum circuit is a sequence of quantum gates and measurements applied to qubits."

]

```
dataset = Dataset.from_dict({  
    "text": training_data  
})
```

```
print("Training examples:", len(dataset))
```

```
# =====
```

```
# 4. Select a small language model
```

```
# =====
```

```
model_name = "distilgpt2"
```

```
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model = AutoModelForCausalLM.from_pretrained(model_name)
```

```
# GPT-style models need a padding token
```

```
tokenizer.pad_token = tokenizer.eos_token
```

```
model.config.pad_token_id = tokenizer.eos_token_id
```

```
print("Model loaded successfully!")
```

```
# =====
```

```
# 5. Tokenize the training data
```

```
# =====
```

```
def tokenize_function(example):
```

```
    return tokenizer(
```

```
        example["text"],
```

```
        truncation=True,
```

```
        padding="max_length",
```

```
        max_length=128
```

```
    )
```

```
tokenized_dataset = dataset.map(
```

```
    tokenize_function,
```

```
    batched=True
```

```
)
```

```
# =====
```

```
# 6. Create data collator
```

```
# =====
```

```
data_collator = DataCollatorForLanguageModeling(
```

```
    tokenizer=tokenizer,
```

```
    mlm=False
```

```
)
```

```
# =====
```

```
# 7. Fine-tuning settings
```

```
# =====
```

```
training_args = TrainingArguments(  
    output_dir="./quantum_chatbot",  
    num_train_epochs=3,  
    per_device_train_batch_size=2,  
    learning_rate=5e-5,  
    logging_steps=1,  
    save_strategy="no",  
    report_to="none"  
)
```

```
# =====
```

```
# 8. Fine-tune the model
```

```
# =====
```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=tokenized_dataset,  
    data_collator=data_collator  
)
```

```
print("\nStarting fine-tuning...")
```

```
trainer.train()
```

```
print("\nFine-tuning completed!")
```

```
# =====
```

```
# 9. Chatbot function
```

```
# =====
```

```
def quantum_chatbot(question):
```

```
    prompt = (  
        "Student: " + question +  
        "\nTutor:"  
    )
```

```
    inputs = tokenizer(  
        prompt,  
        return_tensors="pt"  
    )
```

```
    with torch.no_grad():  
        output = model.generate(  
            **inputs,  
            max_new_tokens=60,  
            do_sample=True,  
            temperature=0.7,  
            top_p=0.9,
```

```

        pad_token_id=tokenizer.eos_token_id
    )

response = tokenizer.decode(
    output[0],
    skip_special_tokens=True
)

# Extract tutor response
if "Tutor:" in response:
    response = response.split("Tutor:", 1)[1]

return response.strip()

# =====

# 10. Test the chatbot

# =====

print("\n=====")
print("    QUANTUM AI TUTOR")
print("=====")

questions = [
    "What is a qubit?",
    "What is superposition?",
    "What is a quantum gate?",

```

```

    "What is entanglement?"
]

for question in questions:

    print("\nStudent:", question)

    answer = quantum_chatbot(question)

    print("Tutor:", answer)

# =====

# 11. Interactive chatbot

# =====

print("\n=====")
print("Interactive Chatbot")
print("Type 'exit' to stop.")
print("=====")

while True:

    question = input("\nYou: ")

    if question.lower() == "exit":
        print("Tutor: Goodbye!")

```

break

```
answer = quantum_chatbot(question)
```

```
print("Tutor:", answer)
```

INPUT / OUTPUT:

```
Student: What is a qubit?  
Tutor: A qubit is a qubit in which a qubit is a quantum state, or a qubit is a quantum state, or a qubit is a quantum state, or a quantum state, or a quantum state, or a quantum state.  
  
Student: What is superposition?  
Tutor: Superposition is a superposition of the superposition of a single superposition. A superposition is a superposition of two superposition-superpositions. The two super  
Superposition is a superposition of two  
  
Student: What is a quantum gate?  
Tutor: A quantum gate is a quantum state, which is a state that is a quantum state that is a quantum state. A quantum gate is a quantum state that is a quantum state that is  
  
Student: What is entanglement?  
Tutor: What is entanglement?  
Tutor: What is entanglement?  
Tutor: Entanglement is the probability that a quantum particle is entangled in a classical state. Entanglement is the probability that a quantum state is entangled in a clas  
Tutor: Entang
```

```
Interactive Chatbot  
Type 'exit' to stop.
```

```
You: WHAT IS QUBIT?  
Tutor: QUBIT is a single-channel communication protocol that allows the user to send and receive messages from a single channel. It enables the user to receive and receive n  
  
You: EXIT  
Tutor: Goodbye!
```